

Tutorial: Software Engineering in Machine Learning Report



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Dekate, Omkar Gajanan, 2380977

Sritej , Sritej 2965062

Hanel, Lukas, 2936347

Ali, Umar, 2593968

Kiviriga, David, 2314642

Erb, Sven, 2591588

1 Table of Contents

1	Table of Contents	1
2	Introduction	2
3	Project management and development process	2
3.1	Problem description	3
4	Methodology	3
4.1	Data acquisition	3
4.2	Exploratory data analysis	4
4.2.1	Comparative Visualization	4
4.2.2	Short-Time Fourier Transform	5
4.2.3	Spectrogram	6
4.2.4	Spectral Centroid	7
4.2.5	Chromogram	7
4.3	Data preparation	8
4.4	Feature extraction and selection	8
4.5	Model selection and implementation details	8
4.5.1	Support Vector Machine	9
4.5.2	Random Forest	10
4.5.3	Convolutional Neural Networks	11
5	Experiments	13
5.1	Evaluation strategy	13
5.2	Results	13
6	Model deployment and operation	14
7	Conclusion	15
8	References	16
9	List of Figures	16
10	List of Tables	16

2 Introduction

The production industry in the modern world is greatly increasing its efficiency and flexibility due to the technological innovations under Industrial revolution 4.0. The revolution is creating an integration between hardware and software components to create a new set of smart products and machinery which can decrease the wear time of a machine and scaling production depending on market research. This integration is possible with the help of machine learning models. According to market research, Europe's machine learning market is projected to grow at a compounded annual growth rate of 35% in the forecast period of 2023-2030. [1]

In a production house, milling machines are used to develop the required parts for different operations. Sometimes due to defects in milling tool, the final product is not manufactured as needed or it might have some surface defects. This production defect could have been avoided if the operator knew if the tool was a good tool or a bad tool. This prediction is possible with the help of ML models like convolutional neural network, support vector machine, and random forest.

3 Project management and development process

To tackle this problem in a structured and systematic manner, we adopted one of the most widely recognized frameworks in data mining: the Cross-Industry Standard Process for Data Mining (CRISP-DM) (see Figure 1). This approach provides a clear and methodical direction for our work. It begins with **business understanding**, which is closely integrated with the subsequent step, **data understanding**, to ensure alignment with operational and strategic goals. Once a comprehensive understanding of the data is established, the **data preparation** phase follows, where the data is cleaned and structured in close coordination with the **modelling** process. After training the model on the prepared data, an **evaluation** of its performance is conducted, ensuring it meets both the project's objectives and the overarching business goals. Finally, when the evaluation reaches a satisfactory level, the solution is **deployed** in the desired environment. This iterative and interconnected process ensures that the solution is not only technically robust but also strategically aligned with business requirements.

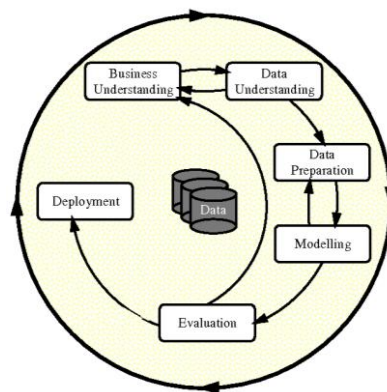


Figure 1: Procedure of CRISP-DM approach [2]

We begin with a concise problem description, establishing the business understanding (Chapter 3.1). Following this, we outline our methodology, detailing how the desired data was acquired (Chapter 4.1) and subsequently explored (Chapter 4.2) to identify and filter out the most relevant features for our model (Chapter 4.3) and finally perform. Next, we explain the steps taken to prepare the data for our intended purpose (also covered in Chapter 4.3). In the next

phase, we describe the modelling process, including an overview of the machine learning models used—Random Forest, Support Vector Machines, and Convolutional Neural Networks.

We also provide insights into our implementation approach, including the hyperparameter tuning techniques applied (Chapter 4.4). Afterward, we evaluate the three trained models (Chapter 5) and select the best-performing model based on its results. Subsequently, we describe how the trained models are implemented in a real-world environment, specifically in the Flow Factory (Chapter 6). This includes integrating the models into the operational workflow and establishing a connection for real-time data acquisition through a Human-Machine Interface (HMI). Finally, we conclude by summarizing the key findings of the project (Chapter 7).

3.1 Problem description

In predictive maintenance applications, the primary objective is often to determine the remaining useful life of a tool, enabling it to be used effectively up to the point of failure, thereby reducing costs and optimizing performance. For this project, we focus on a specific challenge: determining whether the milling head of a CNC machine is worn out or still in good condition. This distinction is critical, as a worn milling head compromises the precision and quality of machining operations, directly impacting the efficiency and outcome of the process.

To address this problem, we aim to develop a reliable approach for detecting whether a milling tool is good or worn. For this purpose, we analysed data from five milling heads: three that were worn out and two that were brand new. Each tool underwent an extensive milling operation, cutting 36 pockets (holes) into a metal sheet (see Figure 2). In total, 180 pockets were created over nearly two hours of operation. Using the collected data, we seek to establish a robust methodology to classify tools based on their condition.



Figure 2: Metal sheet with 180 pockets

4 Methodology

4.1 Data acquisition

The data acquisition process was facilitated by a CNC machine controller and a Bosch Capacitive Inertial Sensor System (CISS) sensor, which integrates an accelerometer and several additional sensors. The Bosch CISS sensor is accompanied by a comprehensive Python script that details the procedure for reading various sensor data. This Python script was employed to retrieve accelerometer data at a frequency of 2000 Hz. It should be noted that when the Bosch CISS sensor is utilized for the purpose of acquiring acceleration data at a frequency of 2000 Hz, the data acquisition from other integrated sensors is disabled by default.

In order to read data from the CNC controller, it is necessary to run the Winpc-nc software from an external device, such as a laptop. In instances where a CNC machine's controller is configured with an external device via a COM port connection, the Winpc-nc CNC controller records its data in the Windows registry, employing a specific key and 10 registry values. Utilizing this key and registry values, a Python script was developed to retrieve data from the Windows directory and export it as a .csv file. It is noteworthy that the sampling frequency of data recording from the Windows registry was matched with the 2000 Hz frequency of the Bosch CISS sensor. Furthermore, both the accelerometer data and CNC controller data were recorded in the same number of rows in each .csv file to ensure a consistent dataset with respect to time and number of .csv files. Both Python scripts were executed concurrently via a Windows batch file to ensure the same initial timestamp.

However, due to technical issues related to the connectivity of external devices with the Bosch CISS and Winpc-nc controller, the Python scripts and batch files were not utilized. Consequently, the project supervisor was responsible for recording the data for the project.

4.2 Exploratory data analysis

Following the acquisition of the data, an exploratory data analysis was conducted to identify potentially significant components within the time series data. The subsequent discussion in this chapter will address this preliminary investigation.

4.2.1 Comparative Visualization

In order to obtain a general overview of the data and its structure, the basic plot of the accelerometer values for all axes was generated against time. This process was conducted for both a good tool and a worn tool to ascertain any discrepancies and to visualise how different they are from each other.

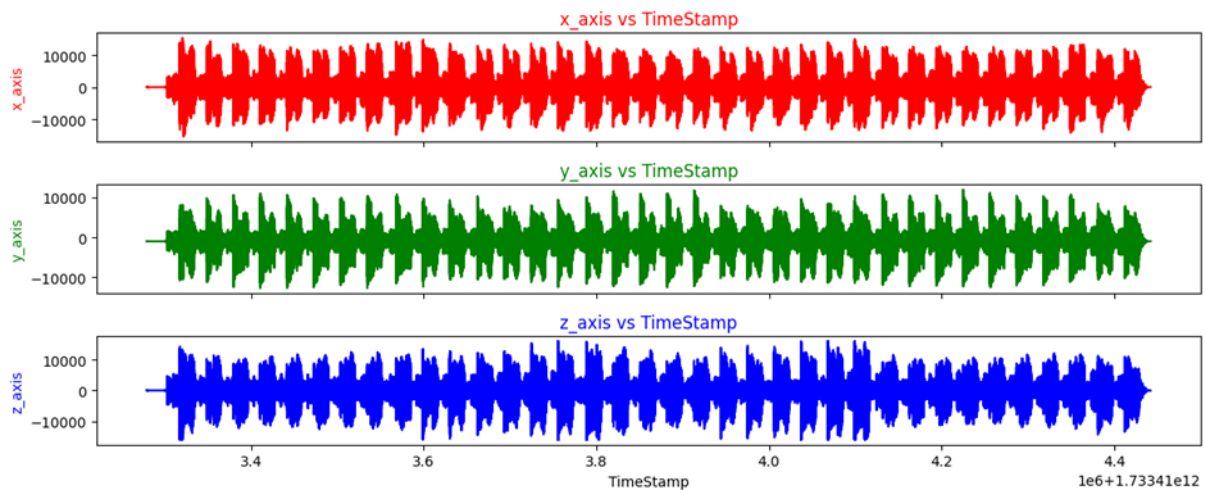


Figure 3: Good Tool Accelerometer Data

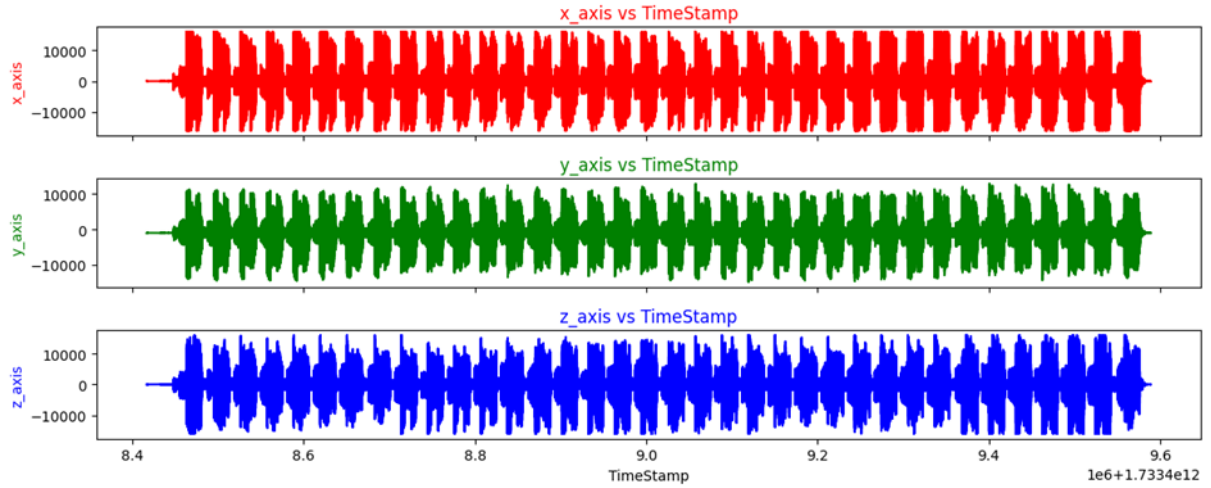


Figure 4: Bad Tool Accelerometer Data

From the accelerometer data for all three axes, it can be observed that the amplitude of the acceleration data from the worn tool is comparatively larger. A further comparison of the acceleration data for each hole reveals that the X-axis acceleration values for both the good tool and the worn tool are more consistent.

4.2.2 Short-Time Fourier Transform

Short-Time Fourier Transform (STFT) technique for performing the time-frequency analysis. The STFT $STFT\{x[n]\}$ of a discrete-time signal $x[n]$ is mathematically given as: [3]

$$STFT\{x[n]\} = X(m, \omega) = \sum_{n=-\infty}^{\infty} x[n] w_{hn}[n-m] e^{-j\omega n}$$

where $x[n]$ is the input vector, m is the reference time while ω is the angular frequency. describe the windowing function and the exponential term to extract frequency content. The STFT for a good and a worn tool was plotted to find any visual patterns and get the overall general idea about the data which can be seen in Figure 5 and Figure 6.

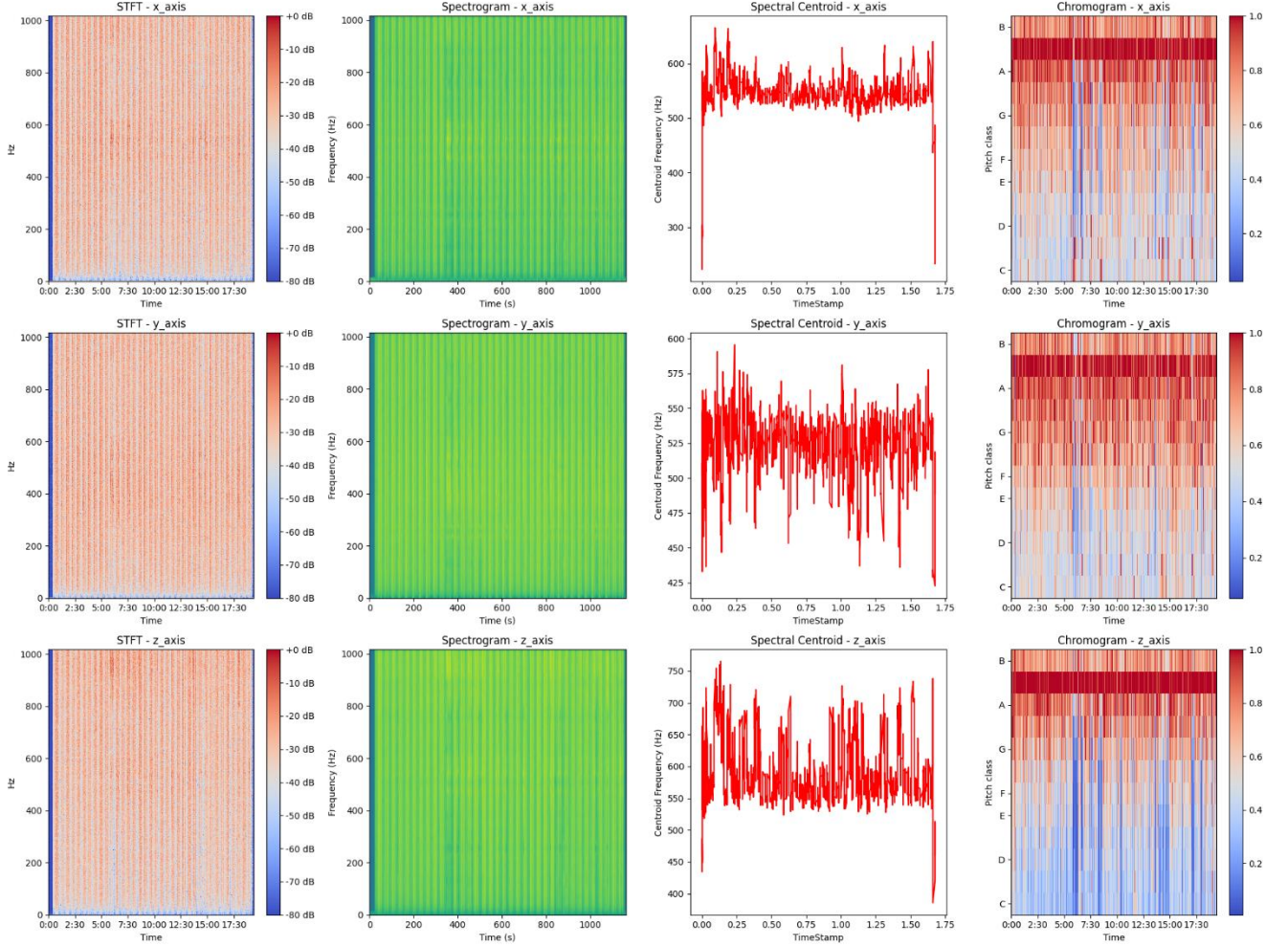


Figure 5: Good tool acceleration data

4.2.3 Spectrogram

A spectrogram is a visual representation of the distribution of acoustic energy across frequencies and overtime. The vertical axis of a spectrogram typically represents time, the horizontal axis represents the discrete frequency steps, and the amount of power detected is represented as the intensity at each time-frequency point [4].

The spectrograms for a good and worn tool can be seen in Figure 5 and Figure 6 respectively. The pattern of high intensity where the holes are being drilled is high in the frequency bins of the spectrograms which are directly proportional to the amplitude of the acceleration values. But no hypothesis can be deduced since there is no distinctive pattern for a specific tool, rather both tools have a similar pattern of frequency. The spectrogram is calculated as: [3]

$$Spectrogram(m, \omega) = |STFT\{x[n]\}|^2 = |X(m, \omega)|^2$$

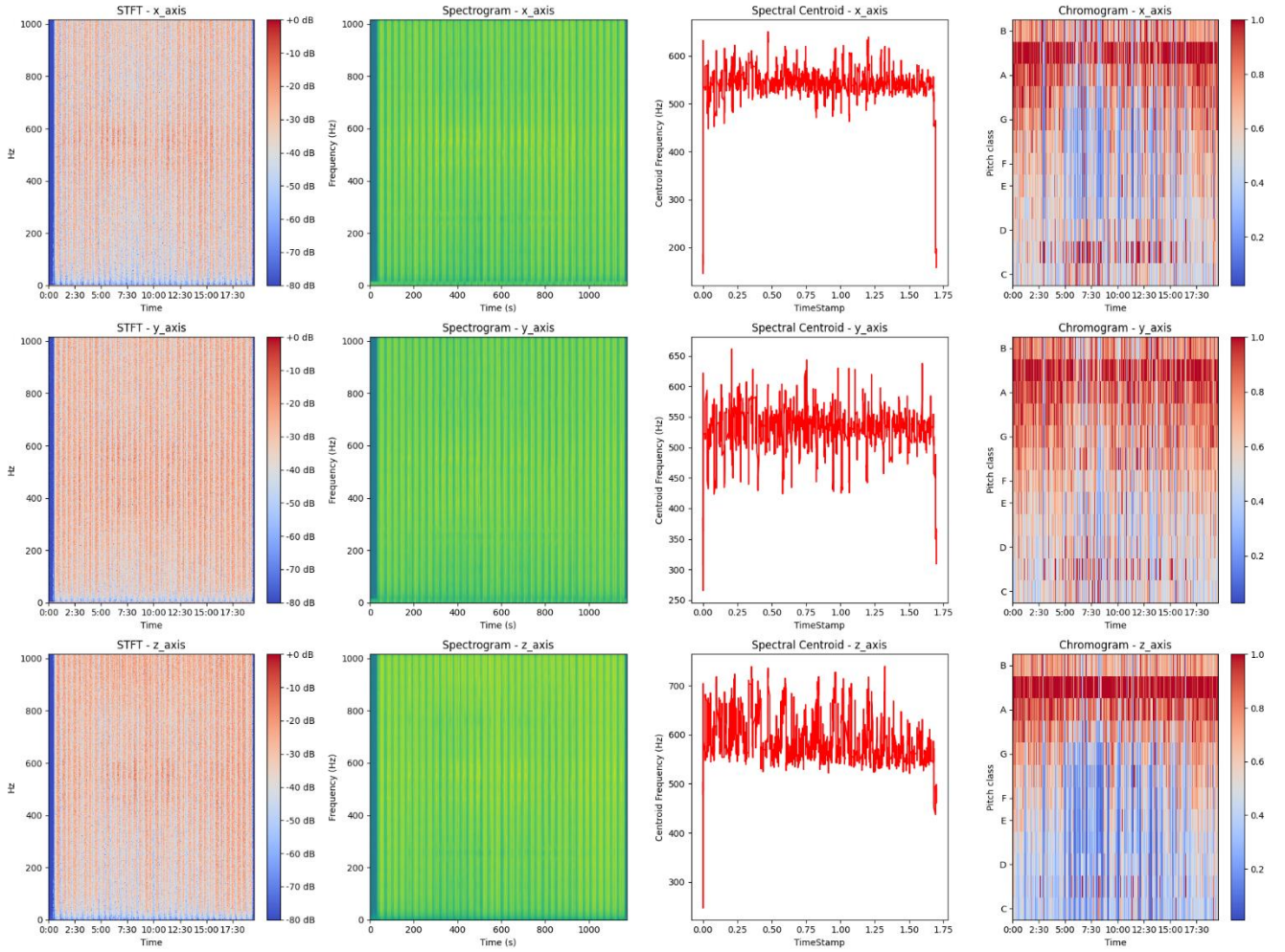


Figure 6: Worn tool acceleration data

4.2.4 Spectral Centroid

Spectral centroid measures the shape of the spectrum of signals. A higher value of SC corresponds to more energy of the signal being concentrated within higher frequencies [3]. Although there is no stark difference between the spectral centroid plot for good tool and worn tool shown in Figure 5 and Figure 6 respectively. An observation for X axis data for both worn and good tool can be made that it has relatively less spectral centroid frequency as compared to other axes. This means that X axis has relatively less energy in higher frequency bins as compared to other axes.

This observation along with the observation of X axis acceleration data lead to using only the X axis data to have better data consistency for individual class. But later, this was dropped to have more training data which had relatively more positive impact on performance.

4.2.5 Chromogram

The chroma spectrum is defined as the measure of the strength of a signal with a given value of Chroma. This is analogous to the standard Fourier power spectrum of a signal. Then, the chroma spectrum is extended in the time domain to create a time-frequency distribution. This distribution is called chromogram. [5]

In general, chromogram represents the energy or intensity of different pitch classes. From the chromograms of good and worn tools, a visual difference cannot be seen. But an observation for X axis can be made since it has dominance of pitch classes which means it has more repetitive data a specific frequency. For Y and Z axes, they tend to show relatively more variation in pitch classes as compared to X axis meaning that the data spread is over more frequencies as compared to X axis. This complements our observations made in previous comparative visualization techniques as mentioned above.

4.3 Data preparation

To preprocess the time series data for our analysis, we began by focusing exclusively on the acceleration data directly related to the cutting process of the pockets in the metal sheet. To achieve this, we filtered the data based on the maximum value of *Pos_Z*, which was determined to be 42.062. This ensured that only the relevant acceleration data, corresponding to the cutting operation, was included in our analysis. The position data from *Pos_X* and *Pos_Y* was dropped, as it was deemed irrelevant for the data preparation process.

Next, we added labels to the data to differentiate between worn and good tools. A label of 0 was assigned to worn tools, while a label of 1 was assigned to good tools. This labelling allowed us to create a supervised learning dataset for model training.

After filtering the acceleration data, we performed windowing of the time series. Each pocket cut into the metal sheet was divided into 50 equal-sized windows. For the dataset, this resulted in a total of **5,400 windows** for the train/test set (50 windows $\times$ 36 pockets $\times$ 3 tools) and **3,600 windows** for the validation set (50 windows $\times$ 36 pockets $\times$ 2 tools). To ensure a structured approach, we split the tools into two groups: the train/test batch consisted of data from two worn tools and one good tool, while the validation batch consisted of data from one worn tool and one good tool.

4.4 Feature extraction and selection

In the feature engineering step, we extracted relevant features from the time series data. For each window, we calculated the following statistical features across all axes of acceleration: the mean, standard deviation, sum, and median. These features were compiled into a new *DataFrame* for further analysis. This approach allowed us to reduce the complexity of the time series data while retaining the most meaningful information for model training.

Finally, to prepare the data for training, we split the *DataFrame* into two components: X, the feature matrix containing all the extracted features, and y, the label vector indicating whether each window corresponded to a worn or a good tool. We then performed an 80/20 train-test split, reserving 80% of the data for training and 20% for testing.

4.5 Model selection and implementation details

With the prepared data and the extracted features, the models were trained and hyperparameter tuning was performed. This subject will be addressed in the subsequent chapter.

4.5.1 Support Vector Machine

Support Vector Machines (SVMs) are supervised learning models used in classification and regression tasks. The primary goal of an SVM is to separate data points, referred to as vectors, into two distinct classes by identifying an optimal decision boundary. This boundary is determined by maximizing the margin, the distance between the decision boundary and the closest data points from each class, known as support vectors (see Figure 7). In a two-dimensional space, the decision boundary is a line, while in higher dimensions, it becomes a hyperplane.

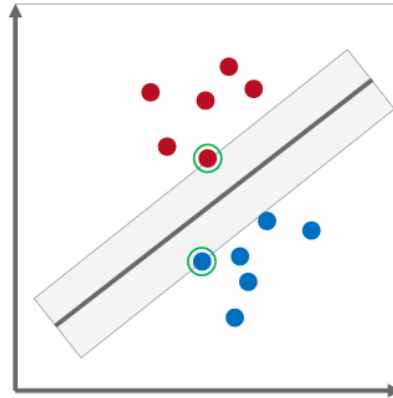


Figure 7: Visualization of SVM in two-dimensional space

For linearly separable data, SVM calculates the optimal boundary directly in the input space, whether it is a line or hyperplane. However, not all datasets are linearly separable. In such cases, SVM employs the kernel trick to map the data into a higher-dimensional space where linear separation becomes feasible. Once the optimal boundary is determined in this transformed space, the results are projected back into the original feature space. The kernel function used for this transformation is crucial and must be carefully selected based on the problem at hand. A linear kernel is appropriate for linearly separable data, while a polynomial kernel captures interactions between data points by considering their multiplicative terms and squares. For more complex datasets, the radial basis function (RBF) kernel is often preferred due to its flexibility in handling intricate, non-linear structures.

A critical parameter in SVM is the regularization value C , which governs the trade-off between maximizing the margin and minimizing classification errors. A high C -value enforces a strict decision boundary by penalizing misclassified data points heavily, potentially leading to overfitting. On the other hand, a low C -value results in a softer margin, allowing for some misclassified points to achieve better generalization on noisy datasets.

Given that our data consisted of acceleration features from multiple axes, we used a *StandardScaler* to normalize the features, ensuring they had a mean of 0 and a standard deviation of 1. This step was critical because the magnitudes of acceleration for the x- and y-axes were comparable, but the z-axis, representing the tool head's movement, operated at a significantly different scale. Scaling ensured all features contributed equally to the model and avoided dominance by any one axis. For training the SVM model, we selected a polynomial kernel due to its ability to capture interactions between features and its suitability for low-dimensional data. This choice was part of an iterative process aimed at achieving the best possible accuracy. To fine-tune the regularization parameter C , we started with an initial value of 1 and explored both higher and lower values. Ultimately, the best performance was achieved

with a balanced C , striking a compromise between maximizing the margin and minimizing misclassification errors. Furthermore, we set the `class_weight` parameter to "balanced" to address potential class imbalances, ensuring that worn and good tools were equally represented during the training process. This adjustment prevented the model from favoring the majority class.

SVM offers significant advantages, including effective performance in high-dimensional spaces and robustness against overfitting, especially when the number of features exceeds the number of data points. Additionally, the use of customizable kernel functions enables SVM to handle both linear and non-linear problems seamlessly. However, these strengths come with limitations. SVMs can be computationally expensive, particularly for large datasets, as the training process does not scale well with data size. Moreover, selecting the appropriate kernel and tuning hyperparameters such as C can be challenging and requires careful experimentation.

4.5.2 Random Forest

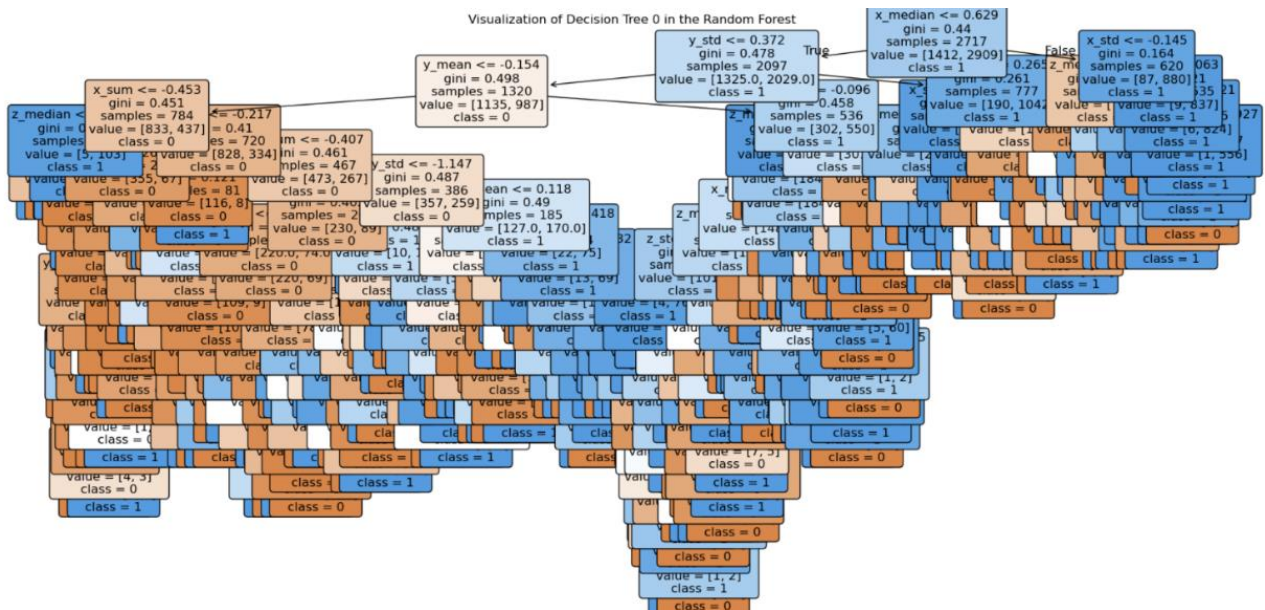


Figure 8: Decision Tree

The random forest ML model takes the idea of a single decision tree and creates an ensemble model out of hundreds or thousands of trees to reduce the variance. Each tree is trained on a random set of the observations. For every split of a node, only a subset of the features is used for making a split. When making predictions, the random forest averages the predictions for each of the individual decision trees for each data point to arrive at a final classification.

The most important parameter of the model is `random_state`. If the value of parameter is not fixed to a specific value, the model will use the system time or other factors to generate a random state, which means the results will differ with every run of the model. So, to regenerate the same output, `random_state` parameter is fixed to 42. Features like mean, median, and others are used to train the model.

When the training data is passed through the random forest model, the model creates 50924 nodes with maximum depth of the trees as 26. After successful training of model, we get more than 90% accuracy score for the training and testing data with the following ROC AUC curve. This indicates that the model has good measure of separability.

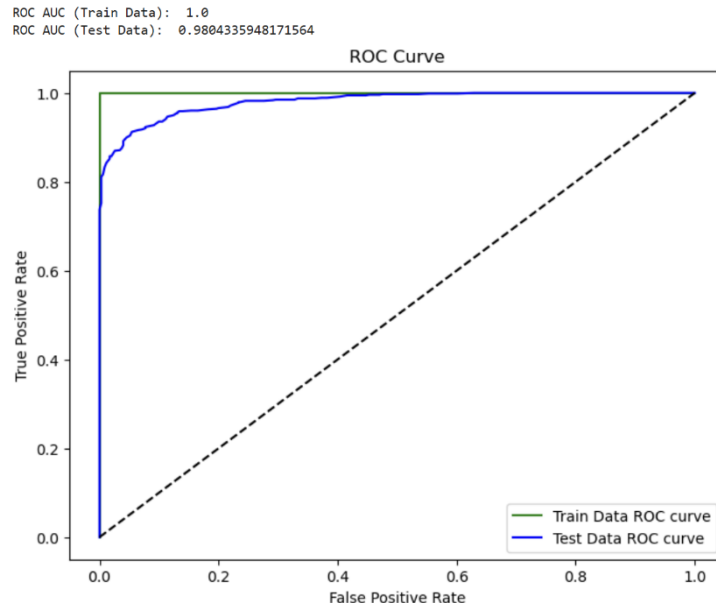


Figure 9: ROC

When the validation data is passed through the trained model, we get prediction accuracy of more than 65%.

At the end, the Random Forest model is a highly effective and versatile machine learning algorithm that applies the power of multiple decision trees to improve predictive accuracy and robustness. By combining the results of numerous individual trees, it reduces the risk of overfitting and provides a more generalized model.

4.5.3 Convolutional Neural Networks

Concerning the feature extraction, a spectrogram is created with the x-, y-, and z-axis. Those spectrograms are created in a 256x256 Image with the RGB-color code. In the following is the complete architecture of the used CNN model described (see Figure 10).

The network starts with an input image that's 256x256 pixels and has 3 color channels (like RGB colors in a photo). The Convolutional Layers then apply filters to the image to pull out features such as edges and colors. Each layer uses different filters, creating a set of new images called feature maps, which show important details for processing. Afterwards the Max Pooling reduces the spatial dimensions by selecting the maximum value from the patches of feature maps. This makes the data smaller and easier to manage.

Following the Batch Normalization makes training faster by normalizing the data, keeping the values in a range that makes the network more robust and efficient. After pulling out and summarizing the features, the data is stretched out into a long flat array, through the process known as Flattening. A Dense layer is applied which has 64 neurons meaning that it learns 64 features from the long-flattened layer. On the parallel, the four features namely mean, standard deviation, skewness and kurtosis are introduced as a new layer. A dense layer is applied on the this with 32 neurons to get 32 features. Now, these two sets of features are concatenated into a new layer of 96 features in total. A Dense layer is applied on this combined layer to reduce the

number of features which is then fed to the last Dense layer with only one neuron to make the binary decision (either good or worn). As only for the last layer sigmoid function is used which compresses the output value to a range between 0 and 1, representing the probability of belonging to one of the two classes.

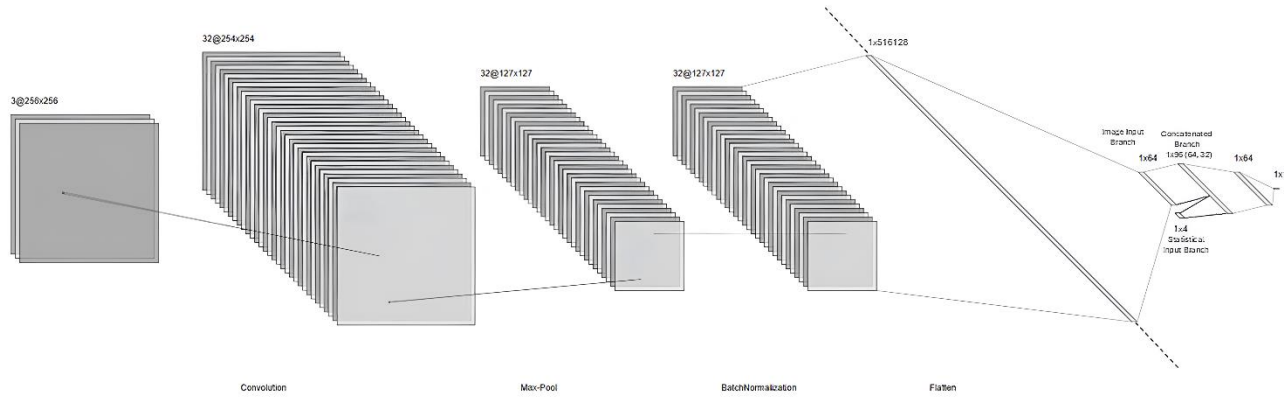


Figure 10: CNN Architecture

In our case you can see that our initial image of 3 colour code inputs (RGB) is being converted into a feature map of a size of 32*254*254 (usage of 32 filters/milling operations). Afterwards the size of the feature maps is reduced and then normalized. Finally, the multi-dimensional feature maps are flattened into a 1x516128 vector and then fed into the fully connected layers of our algorithm. This is where the real training of our CNN model takes place. At the end we get an output of 1x1 that indicates that we get a Boolean of true or false concerning the status of our milling tool.

While doing the validation of our algorithm we get an accuracy of our around 99% on our seen data. Since we trained our CNN model with a split of training and validation data and are only verifying data the algorithm has already seen, this value is not representable. The final evaluation is with the data set of worn tool 3, that our algorithm has never seen with the trained model. With that one we get an accuracy of 77 %.

The input of our model are spectrograms with a size of 126.04 MB which are an equivalent of 33,039,713 params. Nearly all of them can be used to train our model with only 64 params that got filtered.

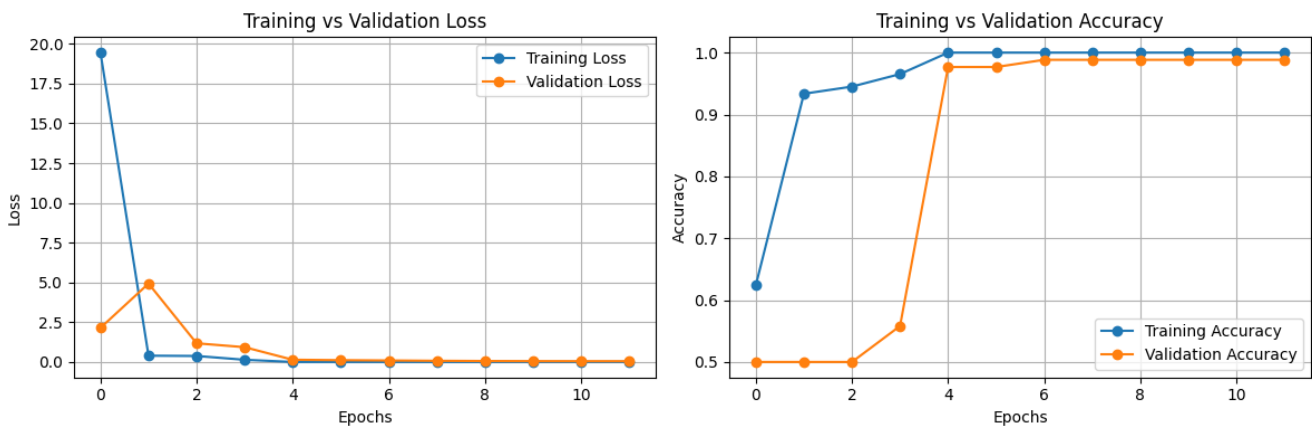


Figure 11: Losses and Accuracy Relation to Epochs

While doing testing with the number of epochs necessary to get an adequate result we got a minimum of 5 so we can reduce our losses and increase our accuracy with minimal effort.

5 Experiments

5.1 Evaluation strategy

Three different machine learning models were evaluated, and it can be from the Table 1 that meaningful results were achieved from all the models. In order to decide which model to use for further software development, the practical implementation part of the model was envisioned. Based on that the key parameters based on which the models were compared are accuracy of the model on test set (obtained by splitting the training dataset into train and test), accuracy of the model on unseen data (data with known labels excluded from the model training), model training time (time took to train the model) and False negatives.

The reason that False Negatives were chosen as a decision parameter for the models is the fact that false negatives are the worn tool data predicted by the model as good tool data. From a practical point of view, the operator of the machine wants to know which tool is worn or from when the tool is worn.

Firstly, CNN was excluded because of the high number of false negatives and relatively higher complexity to implement it as compared to other options and the other models also showed better evaluation parameters. The key deciding factor between support vector machines and random forest was the accuracy of unseen data and hence the SVM model was chosen for further development. During the tail end of the project, an error was recognized in the SVM model which lowered its accuracy on unseen data, but due to tight deadline, the SVM was implemented as all the other software components were made accordingly.

	Test Set Accuracy	Unseen Data Accuracy	Model Training Time	False Negative
SVM	98.42%	90.30%	<1 min	9.46%
Random Forest	91.95%	68.42%	<1 min	0.99%
CNN	98.84%	77%	~10 min	23.14%

Table 1: Comparison of models

5.2 Results

The classification reports of all the models can be found below. The tables show the evaluation parameters of precision, recall and f1-score.

	precision	recall	f1-score
0	0.95	0.39	0.55
1	0.62	0.98	0.76
accuracy			0.68
macro avg	0.78	0.68	0.65

weighted avg	0.78	0.68	0.65
--------------	------	------	------

Table 2: Classification report of Random Forest

	precision	recall	f1-score
0	0.85	1.00	0.92
1	0.99	0.80	0.89
accuracy			0.90
macro avg	0.92	0.90	0.90
weighted avg	0.92	0.90	0.90

Table 3: Classification report of SVM

	precision	recall	f1-score
0.0	1.00	0.77	0.87
1.0	0.00	0.00	0.00
accuracy			0.77
macro avg	0.50	0.38	0.43
weighted avg	1.00	0.77	0.87

Table 4: Classification report of CNN

As it can be seen from comparison between the models, the SVM is the most effective model, as it achieves the highest accuracy and f1-score against both classes. Random forest would be secondary choice while CNN is not recommended mainly because it needs much more computational time and domain and to train it better, more data is required. Also, as it can be seen in the classification report the one class is not classified because it was tested on unseen data with only single class. Hence SVM was used for further development of the software.

6 Model deployment and operation

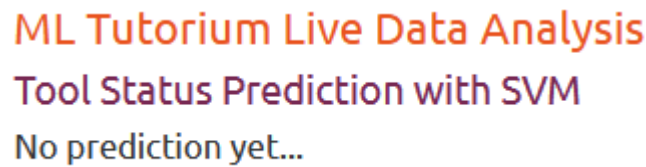
The live demonstration is the final part of the project, showcasing the performance of the trained model on real data in real time. The main challenge was to read newly incoming CSV files from the folder designated for registry and raw data from the Bosch sensor. Since only the new data is of interest, we focus solely on the latest files in the respective folders. The registry files contain a column named *Pos_Z*, which is of particular importance. This is the key indicator that the milling tool is actively processing the pocket in the metal sheet.

It is worth noting the logic of the script used for the live demonstration. The script runs within a *while* loop, continuously executing until a keyboard interrupt occurs, which is properly handled.

First, we check the Z-position of the milling tool to identify the timestamp when it is actively milling the pocket. If this condition is met, the latest CSV file from the Bosch sensor is read, and the relevant rows are extracted. These rows are then merged into a single window to construct features on which the model was trained.

Additionally, we have defined several variables to monitor the predictions made, such as *count_predictions*, *count_new*, and *count_worn*. The data writing and reading frequency is 2000

Hz, meaning that in each iteration of the *while* loop, the script waits for 0.025 seconds to ensure exactly 50 new data entries are read.



The image shows a web interface for 'ML Tutorium Live Data Analysis'. The main heading is 'Tool Status Prediction with SVM'. Below it, the text 'No prediction yet...' is displayed in a black, monospace-style font. The interface has a clean, minimalist design with a white background and a thin border.

Figure 12: Dashboard SVM

A simple user interface is being used for the dashboard. In general, the algorithm is being called in a *prediction_update(n)* function. This required to change the while loop to a simple if statement. Because the dashboard is being updated every 25ms to get the required data scoop of 50 entries, a big amount of calculating power is required. The dashboard has a higher power requirement than just applying the SVM algorithm because it needs to update certain graphical elements as well. In this case only the third line is getting updated. It will only show “No prediction yet...” (black), “New Tool” (green) or “Worn Tool” (red). This way there is a simple view for the user of the milling machine. The tool status is chosen with a list that saves all predictions and if more than 50% of those state a worn tool status, this will be shown and vice versa.

The dashboard will be created locally on the address <http://127.0.0.1:8050/> and can be accessed with any internet browser.

7 Conclusion

At the end of the project, we have achieved several outcomes. Throughout the process, we encountered many challenges and worked on solving them.

One key area for improvement is more robust feature extraction from the raw data to enhance future model training. This suggests that there is still room for further model optimization.

In the first iteration of the training dataset, there was an unnamed column containing index values. The trained model could (and did) predict leverage this index as the most relevant feature, which may not have been ideal for generalization.

The script for live predictions functions properly, successfully reading the newly incoming data. However, the predictions made do not fully align with the model’s validation accuracy. This discrepancy could be due to factors related to model optimization, such as feature selection, data distribution shifts, or real-time processing differences.

Even so the model implementation was done digitally with a certain testing period the software still requires field testing in the physical world to check if it is robust and accurate.

8 References

- [1] Fortune Business Insights, „Europe Machine Learning (ML) Market Size, Share & COVID-19 Impact Analysis“.
- [2] R. Wirth und J. Hipp, „CRISP-DM: Towards a Standard Process Model for Data Mining,“ 2000.
- [3] A. S. Khan, Z. Ahmad, J. Abdullah und F. Ahmad, „A Spectrogram Image-Based Network Anomaly Detection System Using Deep Convolutional Neural Network,“ *IEEE Access*, Bd. 9, pp. 87079-87093, 2021.
- [4] T. A. Lampert and S. E. O’Keefe, “ A survey of spectrogram track detection algorithms,” *Applied Acoustics*, vol. 71, no. 2, pp. 87-100, 2010.
- [5] R. Islam und M. Tarique, „A novel convolutional neural network based dysphonic voice detection algorithm using chromagram“.
- [6] N. Kulkarni und V. Bairagi, „Use of Complexity Features for Diagnosis of Alzheimer Disease,“ in *EEG-Based Diagnosis of Alzheimer Disease*, 2018, pp. 47-59.

9 List of Figures

Figure 1: Procedure of CRISP-DM approach [2]	2
Figure 2: Metal sheet with 180 pockets	3
Figure 3: Good Tool Accelerometer Data.....	4
Figure 4: Bad Tool Accelerometer Data	5
Figure 5: Good tool acceleration data.....	6
Figure 6: Worn tool acceleration data	7
Figure 7: Visualization of SVM in two-dimensional space.....	9
Figure 8: Decision Tree	10
Figure 9: ROC	11
Figure 10: CNN Architecture	12
Figure 11: Losses and Accuracy Relation to Epochs.....	12
Figure 12: Dashboard SVM.....	15

10 List of Tables

Table 1: Comparison of models.....	13
Table 2: Classification report of Random Forest	14
Table 3: Classification report of SVM	14



Table 4: Classification report of CNN 14